

Лабораторная работа №3

Операционные системы

Игнатенко Д. Б.

10 марта 2026

Российский университет дружбы народов, Москва, Россия

Информация

- Игнатенко Денис Беньяминович
- Студент группы НПИбд-01-25
- Российский университет дружбы народов
- 1032252476@rudn.ru



Цель работы

Научиться оформлять отчёты с помощью легковесного языка разметки Markdown.

Задание

- Сделать отчёт по предыдущей лабораторной работе в формате Markdown
- Предоставить отчёты в 3 форматах: pdf, docx и md

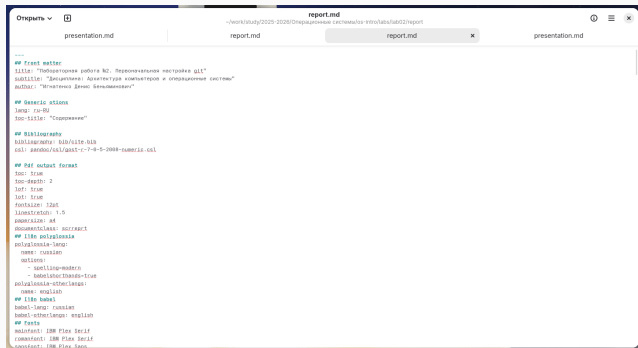
Выполнение лабораторной работы

Отчёт по лабораторной работе №2 я изначально писал с использованием Markdown.

В данной работе описываю процесс создания этого отчёта.

Заполнение YAML-шапки

Открываю файл report.md в текстовом редакторе и заполняю YAML-шапку: title, subtitle, author.



```
---
# Front matter
title: "Политическая работа №2. Персональные настройки gll"
subtitle: "Дисциплина: Архитектура компьютеров и операционные системы"
author: "Игнатенко Денис Вячеславович"

# Generic options
lang: ru-RU
toc-title: "Содержание"

# Bibliography
bibliography: bib/bibtex-bib
col: aarndt/cal/gast-c-7-8-5-2008-cameria-cal

# Pdf output format
toc: true
toc-depth: 2
lof: true
lot: true
fontsize: 12pt
linestretch: 1.5
papersize: A4
documentclass: a4rreport

# Like polyglossia
polyglossia-lang:
  name: russian
  options:
    - spelling=modern
    - babelshorthands=true
polyglossia-ethniclang:
  name: english

# Like babel
babel-lang: russian
babel-ethniclang: english

# Fonts
mainfont: IBM Plex Serif
romanfont: IBM Plex Serif
monofont: IBM Plex Mono
```

Рис. 1: Заполнение YAML-шапки

Написание цели, задания и теории

Формирую основные разделы: цель работы, задание, теоретическое введение с таблицей команд git.

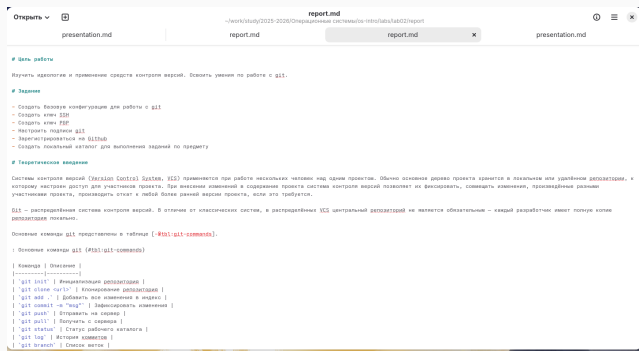
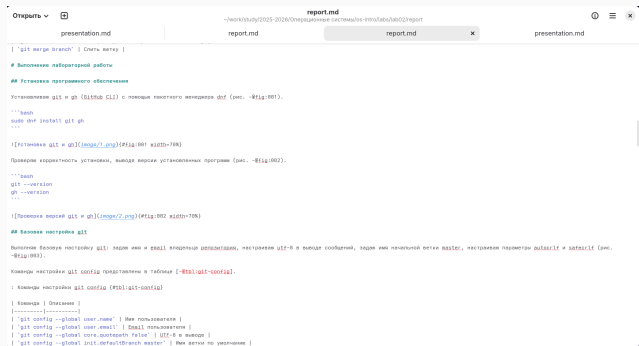


Рис. 2: Цель, задание и теоретическое введение

Описываю выполнение работы с текстом, блоками кода и ссылками на скриншоты.



```
report.md
~work/study/2025-2026/Операционные системы/operating-systems/report

presentation.md | report.md | report.md x | presentation.md

| 'git merge branch' | Ссылка на скриншот |

# Выполнение лабораторной работы

## Установка программного обеспечения

Установили git и gh (GitHub CLI) с помощью пакетного менеджера dnf (рис. -@fig:001).



```
'''bash
sudo dnf install git gh
'''
```



[Установка git и gh](image/1.png){#fig:001 width=700}

Проверили корректность установки, выведем версии установленных программ (рис. -@fig:002).



```
'''bash
git --version
gh --version
'''
```



[Проверка версий git и gh](image/2.png){#fig:002 width=700}

## Базовые настройки git

Выполнив базовые настройки git: задан имя и email владельца репозитория, настроили git-8 на выводе сообщений, задан имя начальной ветки master, настроили параметр autocrlf и safecrlf (рис. -@fig:003).

Команды настройки git config представлены в таблице [-@tbl:git-config].

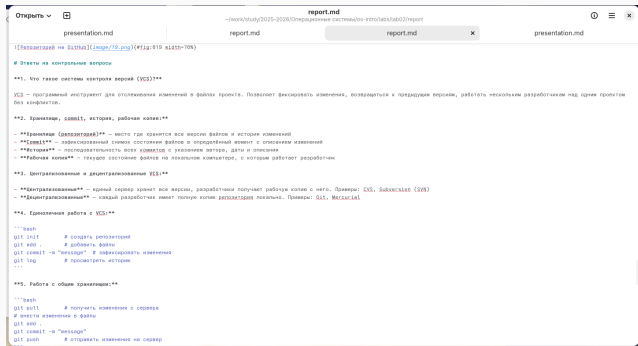
: Команды настройки git config {#tbl:git-config}

| Команда | Описание |
| :--- | :--- |
| 'git config --global user.name' | Имя пользователя |
| 'git config --global user.email' | Email пользователя |
| 'git config --global core.quotePath false' | UTF-8 в выводе |
| 'git config --global init.defaultBranch master' | Имя ветки по умолчанию |
```

Рис. 3: Раздел выполнения работы

Написание контрольных вопросов

Формирую ответы на контрольные вопросы с использованием форматирования Markdown.



```
[[Рекомендаций == <img alt="Git logo" data-bbox="345 295 385 315"/>]](image/19.png)(?f1a:619 width=70%)

# Ответы на контрольные вопросы

**1. Что такое системы контроля версий (VCS)?**

VCS – программный инструмент для отслеживания изменений в файлах проекта. Позволяет фиксировать изменения, возвращаться к предыдущим версиям, работать нескольким разработчикам над одним проектом без конфликтов.

**2. Хранилище, commit, история, рабочая копия:**

- **Хранилище (репозиторий)** – место где хранятся все версии файлов и истории изменений
- **Commit** – зафиксированный снимок состояния файлов в определенный момент с описанием изменений
- **История** – последовательность всех коммитов с указанием автора, даты и описания
- **Рабочая копия** – текущее состояние файлов на локальном компьютере, с которым работает разработчик

**3. Централизованные и децентрализованные VCS:**

- **Централизованные** – единый сервер хранит все версии, разработчики получают рабочую копию с него. Примеры: CVS, Subversion (SVN)
- **Децентрализованные** – каждый разработчик имеет полную копию репозитория локально. Примеры: Git, Mercurial

**4. Ежедневная работа с VCS:**



```
'''bash
git init # создать репозиторий
git add . # добавить файлы
git commit -m "message" # зафиксировать изменения
git log # просмотреть историю
'''
```



**5. Работа с общим хранилищем:**

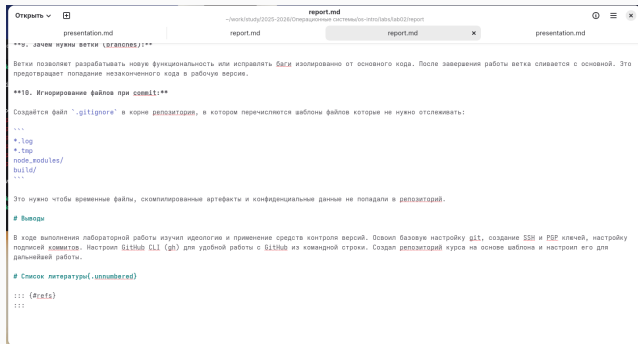


```
'''bash
git pull # получить изменения с сервера
внести изменения в файлы
git add .
git commit -m "message"
git push # отправить изменения на сервер
```


```

Рис. 4: Контрольные вопросы

Завершаю отчёт разделом выводов и списком литературы.



```
Открыть ▾ [icon] --/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report
presentation.md report.md report.md presentation.md

***9. Зачем нужны ветки (branches)***

Ветки позволяют разрабатывать новую функциональность или исправлять баги изолированно от основного кода. После завершения работы ветка сливается с основной. Это предотвращает попадание незаконченного кода в рабочую версию.

**10. Игнорирование файлов при commit**

Создаётся файл '.gitignore' в корне репозитория, в котором перечисляются шаблоны файлов которые не нужно отслеживать:

...
*.log
*.tap
node_modules/
build/
...

Это нужно чтобы временные файлы, скомпилированные артефакты и конфиденциальные данные не попадали в репозиторий.

# Выводы

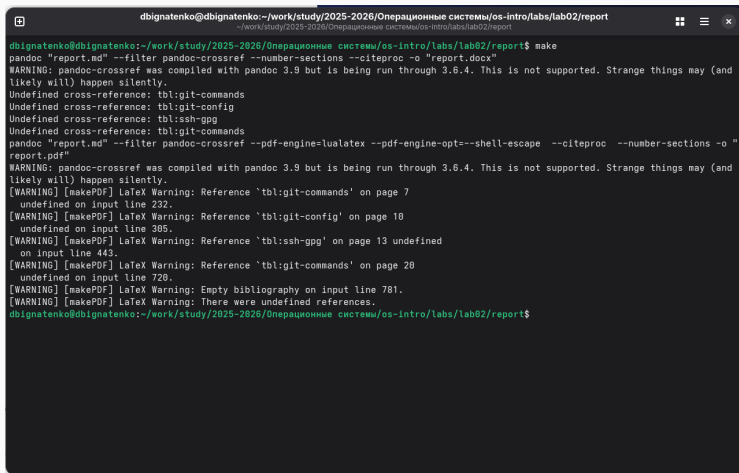
В ходе выполнения лабораторной работы изучил идеологию и применение средств контроля версий. Освоил базовую настройку git, создание SSH и PGP ключей, настройку подписей коммитов. Настроил GitHub CLI (gh) для удобной работы с GitHub из командной строки. Создал репозиторий курса на основе шаблона и настроил его для дальнейшей работы.

# Список литературы(unnumbered)

::: {#refs}
:::
```

Рис. 5: Выводы и список литературы

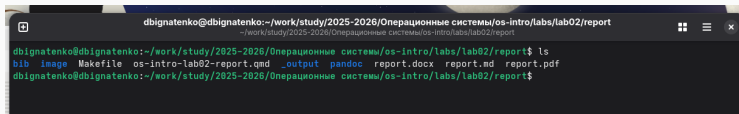
Выполняю компиляцию командой make. Pandoc создаёт report.pdf и report.docx.

A terminal window with a dark background and light text. The window title is "dbignatenko@dbignatenko: ~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report". The prompt is "dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report\$". The user enters the command "make". The output shows the execution of "pandoc 'report.md' --filter pandoc-crossref --number-sections --citeproc -o 'report.docx'". It includes a warning about pandoc-crossref version compatibility and several "Undefined cross-reference" messages for "tbl:git-commands", "tbl:git-config", and "tbl:ssh-gpg". Then, it shows the execution of "pandoc 'report.md' --filter pandoc-crossref --pdf-engine=lualatex --pdf-engine-opt=--shell-escape --citeproc --number-sections -o 'report.pdf'". This also includes a version warning and several LaTeX warnings about undefined references to the same labels on specific pages (7, 10, 13, 28) and an empty bibliography. The terminal ends with the prompt "dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report\$".

```
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report$ make
pandoc "report.md" --filter pandoc-crossref --number-sections --citeproc -o "report.docx"
WARNING: pandoc-crossref was compiled with pandoc 3.9 but is being run through 3.6.4. This is not supported. Strange things may (and
likely will) happen silently.
Undefined cross-reference: tbl:git-commands
Undefined cross-reference: tbl:git-config
Undefined cross-reference: tbl:ssh-gpg
Undefined cross-reference: tbl:git-commands
pandoc "report.md" --filter pandoc-crossref --pdf-engine=lualatex --pdf-engine-opt=--shell-escape --citeproc --number-sections -o "
report.pdf"
WARNING: pandoc-crossref was compiled with pandoc 3.9 but is being run through 3.6.4. This is not supported. Strange things may (and
likely will) happen silently.
[WARNING] [makePDF] LaTeX Warning: Reference 'tbl:git-commands' on page 7
undefined on input line 232.
[WARNING] [makePDF] LaTeX Warning: Reference 'tbl:git-config' on page 10
undefined on input line 385.
[WARNING] [makePDF] LaTeX Warning: Reference 'tbl:ssh-gpg' on page 13 undefined
on input line 443.
[WARNING] [makePDF] LaTeX Warning: Reference 'tbl:git-commands' on page 28
undefined on input line 728.
[WARNING] [makePDF] LaTeX Warning: Empty bibliography on input line 781.
[WARNING] [makePDF] LaTeX Warning: There were undefined references.
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report$
```

Рис. 6: Запуск make

Проверяю содержимое каталога командой `ls` — файлы успешно созданы.



```
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report$ ls
bib image Makefile os-intro-lab02-report.qmd _output pandoc report.docx report.md report.pdf
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report$
```

Рис. 7: Проверка файлов

Выводы

В ходе выполнения лабораторной работы научился оформлять отчёты с помощью легковесного языка разметки Markdown. Освоил компиляцию документов в PDF и DOCX с использованием Pandoc.